

String, Number and Math() function

String -

- Sequence of characters used to represent text
 ↓
A, B, c, a, b, c
- Abc → string, Satyam → string

JS support three ways to create String -

- ① " → string → const a = "Satyam"
- ② ' → string → const b = 'Thakur'
- ③ ` → string → const c = `Shivam`

→ Satyam → no. of characters → length of string

const name = "satyam"

`name.length` → 6

String index →

- strings are zero-indexed.

`const name = 'Satyam'`

↓ ↓ ↓ ↓ ↓ ↓
0 1 2 3 4 5

`name.length` → 6
`name[name.length - 1]` → m
`name[5] = m` (last character)

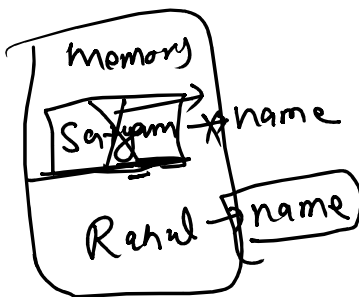
`name[0] ⇒ S`
`name[1] ⇒ a`

Strings are Immutable ✓

→ string cannot be directly changed character by character

`const name = 'Satyam'`

`name[0] = 'R' → X`



`console.log(name) → Satyam`

→ name = "Rahul" ✓

→ you can create complete new strings

Concatenation → Joining strings
↓ use (+)

```
const fname = "Satyam"
```

```
const lname = "Thakur"
```

```
const FullName = fname + " " + lname
```

⇒ Satyam Thakur

Template Literal - [``` ```] → backticks

→ multiline strings, Dynamic string

```
const description = `My name is Satyam  
I'm from bihar  
Sitamarhi`  
console.log(description)
```

Dynamic String -

```
const name = "Shivam"  
const message = `My name is ${name}`  
console.log(message)
```

toUpperCase() / toLowerCase()

const name = "Satyam"

const name = "satyam"

const name: "Satyam"

name.toUpperCase()

2

"SATYAM"

→ Converts string to upper case

```
const name = "SATYAM"
```

name.toLowerCase(),

"Seyam"

→ Convert to lower case

Time()

→ Remove whitespaces from the beginning and end.

```
const name = "Satyam"
console.log(name)
const newName = name.trim()
console.log(newName)
```

Note - trim() does not remove space in the middle

"Suryan" ~~↓~~ "Kumar". trim() X
does not remove

→ trimStart() → only remove from start

→ $\text{trim End } () \rightarrow \text{end}$ ✓

includes() → True/false

→ Checks whether a string contains a particular value.

```
const text = "Javascript is awesome"
console.log(text.includes("awesome")) → True
```

startsWith() →

→ Checks whether a string starts with a specific value ~~or~~ or, not

- return true/false

endsWith() → ends with a specific value or not

```
const text = "Javascript is awesome"
console.log(text.startsWith("Javascript"))
console.log(text.endsWith("awesome"))
```

charAt() → index

→ Returns the character at a particular index

" 0 1 2 3 4 "

index

const name = "citizen"

name.charAt(3) → y

slice() method

→ Extracts part of a string

Syntax -

String-name.slice(start, end)

↑ not include
↓ include

end
Slice(one index)

const text = "JavaScript"

text.slice(0, 4) → Java

text.slice(4, 10) → Script

Negative slice -

text = "JavaScript"

-6 -4
+1
1 3 2 1
- - 5 - - -

text.slice(-6) → Script

Deleting()

substring() -

→ Similar to slice() but has different behavior with negative indexes

text = "JavaScript"



text.substring(0, 4) → Java

note → slice() → supports negative indexes
substring() → treats negative value
as 0

text.substring(-6) → JavaScript

substr() -

→ prefer slice()

→ substr() is a legacy / deprecated method and should generally not be used in new code

`# replace(" ↓ " , " ↓ ")` `# replaceAll(" " , " ")`
 Replace क्या करनी है
 पहली कतार → Replaces all matching occurrences
 → Replaces the first matching occurrence

```
const text = "Hello World. The World is beautifull"
console.log(text.replace("World", "Dipa"))
console.log(text.replaceAll("World", "Dipa"))
```

Split() -

→ Converts a string into an array.

`text = "JavaScript"`
`text.split("")` ; All characters
 Base

```
const text = "Java Script"
console.log(text.split(""))
console.log(text.split(" "))
const text2 = "Java-Script"
console.log(text.split("-"))
const skills = "Js,React, node, next"
console.log(skills.split(","))
```

concat() - Joins strings

`let fname : "Hello"`

```
let fname = "Hello"
let lname = "World"
log(fname.concat(" ", lname))
```

No. of string

Hello World

```
fname.concat("Satyam", " Ap के है ते है ")
```

(Hello Satyam Ap ———)

```
const first = "Hello"
const last = "World"
console.log(first.concat(" ", last))
console.log(first.concat(" ", last, ". ", "My name is Satyam"))
```

Repeat() → Repeats a string a specified No. of times

```
const name = "Satyam"
name.repeat(3)
"Satyam Satyam Satyam"
```

Numbers → numeric value.

age = 25

neg = -

price = 99.99 ✓

negative = -9

⇒ special no. value

Infinity → (10/0) → Infinity - ✓

NaN → Not a Number

± Operators on numbers

↓
already covered ✓

± parseInt() ✓

→ Converts a string into an integer

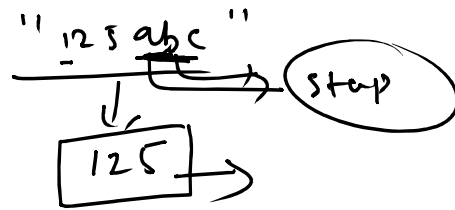
parseInt("25")
↓
25 → Number

parseInt("125abc")
↓ ↓

(125)

It can also stop when
it encounters non.

numeric value



parseFloat()

→ Convert a string into a floating-point number

parseFloat("25.50")

Number.isInteger() → true / false

→ Checks whether a value is an integer
... -2, -1, 3, 2, 1 ←

console.log (Number.isInteger (25)) → true
" " " " (25.5) → false

Number.isNaN()

→ checks whether the value is actually NaN

```

NaN - > not a number
const kuchhBhi = "Satyam"/2
console.log(kuchhBhi)
const kuchhToHai = "Dipa"/4
console.log(NaN == NaN) -> false
console.log(kuchhBhi == kuchhToHai) -> false
console.log(Number.isNaN(kuchhBhi) == Number.isNaN(kuchhToHai)) -> true
console.log(typeof kuchhBhi) -> number
console.log(Number.isNaN(kuchhBhi)) -> true

```

toFixed() ^{num}

→ Formats a number to a specified number of decimal.

price = 99.9567

price.toFixed(2) → 99.95

Return string

```

const price = 99.95678
const toFixedValue = price.toFixed(2)
console.log(toFixedValue)
console.log(typeof toFixedValue)

```

toString() ^{convert} → number into a string

console.log(typeof 100.toString())

↓

string

```
// toString
```

```
let num = 100
let tostr = num.toString()
console.log(typeof tostr)
```

Number Constants

```
//Number constants
console.log(Number.MAX_VALUE)
console.log(Number.MIN_VALUE)
console.log(Number.MAX_SAFE_INTEGER)
```

etc.

Floating point precision

— js can have floating - point precision issue.

```
//floating point issues
console.log(0.1 + 0.2)
// output -> 0.30000000000000004
```

→ This happens because of binary floating-point representation.

Math()

→ math is a built in JavaScript object that provides mathematical constants and function.

```

//Math() object
// Math.max() -> return maximum value
// console.log(Math.max(10,12,13,43,100,19))

// Math.min() -> return minimum value
// console.log(Math.min(10,12,13,43,100,19))

// Math.round() - nearest integer
// console.log(Math.round(4.9))

// Math.floor() -> rounds downward
// console.log(Math.floor(4.7))
// console.log(Math.floor(4.1))
// console.log(Math.floor(-4.7))

// Math.ceil() -> rounds upward
// console.log(Math.floor(4.7))
// console.log(Math.floor(4.1))
// console.log(Math.floor(-4.7))

// Math.trunc() -> remove the decimal portion without rounding
// console.log(Math.trunc(4.6))
// console.log(Math.trunc(4.1))

// Math.abs() -> return the absolute value
// console.log(Math.abs(-10))

// Math.pow() -> returns a number raised to a power
// 2*2*2
// console.log(Math.pow(2,3))

// Math.sqrt() -> return square root
// console.log(Math.sqrt(25))

// Math.cbrt() -> return cube root
// console.log(Math.cbrt(27))

// Math.PI -> PI value
// console.log(Math.PI)

```

Math.random() -

→ Return a random no.
from 0 to 1 → less than 1

from a to b
↓
include
1 → less than
└ → exclude
(Not include)

```
// Math.random()
// console.log(Math.random())

// question :- generate a random integer from 0 to 9

const random = Math.floor(Math.random() * 10)

console.log(random)
```